Complete Beginner Guide to GitHub Self Hosted Runners

GitHub Actions self-hosted runners let you execute deployment pipelines directly on your own cloud servers. This helps you lower build costs and allows your pipelines to safely access internal network resources.

What you'll learn:

1. Downloading and Unpacking the Runner Agent
2. Connecting the Server to Your GitHub Repository
3. Running the Agent as a System Daemon
4. Updating Workflow YAML Files to Use Your Runner

Aman Raj Singh
Cloud Operations Engineer @ iManage
linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Downloading and Unpacking the Runner Agent

To set up a self-hosted runner, log into your cloud server using SSH. Create a new folder dedicated to the runner software to keep files organized. Use curl to download the official Linux runner package from GitHub releases, then extract it using tar.

```
$mkdir actions-runner && cd actions-runner
 curl -o runner.tar.gz -L https://github.com/actions/runner/releases/download/v2.314.0/actions-runner
 tar xzf ./runner.tar.gz
```

GitHub Actions Self-Hosted Runners: Run Pipelines on Your Server

Friday, October 16, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Connecting the Server to Your GitHub Repository

Navigate to your GitHub repository settings under Actions > Runners to generate a setup token. Run the config script in your terminal using this token to register the server. You can also name your runner and assign custom labels for workflow targeting.

```
$./config.sh --url https://github.com/your-username/your-repo --token A123456789EXAMPLETOKEN --name m
```

GitHub Actions Self-Hosted Runners: Run Pipelines on Your Server

Friday, October 16, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Running the Agent as a System Daemon

Running the agent manually with run.sh works for tests, but stops when you exit SSH. Use the service script (svc.sh) to install the runner as a background systemd service. This ensures the runner starts automatically whenever the server boots.

```
$sudo ./svc.sh install
 sudo ./svc.sh start
 sudo ./svc.sh status
```

GitHub Actions Self-Hosted Runners: Run Pipelines on Your Server

Friday, October 16, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Updating Workflow YAML Files to Use Your Runner

Open your workflow file inside the .github/workflows directory in your repository. Change the standard runs-on key from ubuntu-latest to self-hosted. Commit and push the changes to trigger pipeline execution on your target server.

```
$jobs:
  deploy:
    runs-on: [self-hosted, staging]
    steps:
      - uses: actions/checkout@v4
      - run: echo "Deploying on my server!"
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Always install the runner as a system service so it stays online continuously.
+ Use custom tags like 'staging' or 'gpu' to route specific pipeline jobs correctly.
+ Keep server dependencies like Docker or Node.js pre-installed on the host machine.
+ Isolate your runner servers inside private network subnets for better security.
+ Regularly clean up temporary workspace files using automated cleanup steps.

Watch Out For

! Attaching self-hosted runners to public repositories where untrusted users can submit pull requests.
! Running ./run.sh interactively instead of running svc.sh as a system background service.
! Forgetting that registration tokens expire quickly, causing configuration script failures.
! Not installing required CLI tools (like Docker or AWS CLI) on the runner host server.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh